

Project: Automate testing for an E-commerce Checkout Flow

Modules to Test:

- User login
- Product search
- Add to cart
- Apply coupon
- Checkout form
- Order confirmation

Learning Goals:

Write end-to-end automation test cases Test positive and negative scenarios Validate UI messages and form errors Learn basic test reporting Understand how automation helps in real projects.

Suggested Tool:

Playwright or Cypress

What to Submit:

2-page PDF report

Test cases list

Screenshots of automation results

Tool used + short learning notes

Timeline: 1 day

E-commerce Checkout Flow Automation using Playwright

Introduction

Automation testing is a software testing approach where test cases are executed automatically using tools and scripts instead of manual effort. In this project, an end to end E-commerce checkout flow is automated using Playwright to verify key functionalities such as user login, product selection, adding items to cart, applying coupons, completing the checkout form, and validating order confirmation. The purpose of this automation is to ensure that the application works correctly under both positive and negative scenarios while reducing manual testing time and increasing accuracy. This project also helps in understanding how real-world web applications are tested, how UI elements and error messages are validated, and how automation improves efficiency, reliability, and consistency in the software testing process.

Tool Used

- Playwright
- JavaScript
- Browser: Chrome

Modules Tested

1. User Login
2. Product Search *(Not available in demo site)*
3. Add to Cart
4. Apply Coupon *(Not available)*
5. Checkout Form
6. Order Confirmation

Test Cases

ID	Scenario	Result
TC1	Valid Login	Pass
TC2	Invalid Login	Pass
TC3	Add to Cart	Pass
TC4	Checkout Form	Pass
TC5	Order Confirmation	Pass

Positive Testing

- Successful login
- Product added to cart
- Checkout completed
- Order placed successfully

Negative Testing

- Invalid login shows error message

Validation

- UI messages verified
- Form fields validated
- Error messages checked

Create An Ecommerce Testing Application using cmd and playwright:

```
cmd npm install @playwright/test

C:\Windows\system32>npm -v
11.12.1

C:\Windows\system32>mkdir ecommerce-assignment

C:\Windows\system32>cd ecommerce-assignment

C:\Windows\System32\ecommerce-assignment>npm init playwright@latest
Need to install the following packages:
create-playwright@1.17.139
Ok to proceed? (y) y

> npx
> create-playwright

Getting started with writing end-to-end tests with Playwright:
Initializing project in '.'
? Do you want to use TypeScript or JavaScript? ...
  TypeScript
√ Do you want to use TypeScript or JavaScript? · JavaScript
√ Where to put your end-to-end tests? · tests
√ Add a GitHub Actions workflow? (Y/n) · true
√ Install Playwright browsers (can be done manually via 'npx playwright install')? (Y/n) · true
Initializing NPM project (npm init -y)...
Wrote to C:\Windows\System32\ecommerce-assignment\package.json:

{
  "name": "ecommerce-assignment",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

Installing Playwright Test (npm install --save-dev @playwright/test)...
```

```
Administrator: Command Prompt

{"keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

Installing Playwright Test (npm install --save-dev @playwright/test)...

added 3 packages, and audited 4 packages in 7s

found 0 vulnerabilities
Installing Types (npm install --save-dev @types/node)...

added 2 packages, and audited 6 packages in 4s

found 0 vulnerabilities
Writing playwright.config.js.
Writing .github/workflows/playwright.yml.
Writing tests/example.spec.js.
Writing package.json.
Downloading browsers (npx playwright install)...
Downloading Chrome for Testing 147.0.7727.15 (playwright chromium v1217) from https://cdn.playwright.dev/builds/cft/147.0.7727.15/win64/chrome-win64.zip
179.4 MiB [=====] 100% 0.0s
Chrome for Testing 147.0.7727.15 (playwright chromium v1217) downloaded to C:\Users\User\AppData\Local\ms-playwright\chromium-1217
Downloading Chrome Headless Shell 147.0.7727.15 (playwright chromium-headless-shell v1217) from https://cdn.playwright.dev/builds/cft/147.0.7727.15/win64/chrome-headless-shell-win64.zip
111.5 MiB [=====] 100% 0.0s
Chrome Headless Shell 147.0.7727.15 (playwright chromium-headless-shell v1217) downloaded to C:\Users\User\AppData\Local\ms-playwright\chromium-headless-shell-1217
Downloading Firefox 148.0.2 (playwright firefox v1511) from https://cdn.playwright.dev/dbazure/download/playwright/builds/firefox/1511/firefox-win64.zip
113.1 MiB [=====] 100% 0.0s
Firefox 148.0.2 (playwright firefox v1511) downloaded to C:\Users\User\AppData\Local\ms-playwright\firefox-1511
Downloading WebKit 26.4 (playwright webkit v2272) from https://cdn.playwright.dev/dbazure/download/playwright/builds/webkit/2272/webkit-win64.zip
57.6 MiB [=====] 100% 0.0s
WebKit 26.4 (playwright webkit v2272) downloaded to C:\Users\User\AppData\Local\ms-playwright\webkit-2272
Downloading FFMpeg (playwright ffmpeg v1011) from https://cdn.playwright.dev/dbazure/download/playwright/builds/ffmpeg/1011/ffmpeg-win64.zip
1.3 MiB [=====] 100% 0.0s
FFmpeg (playwright ffmpeg v1011) downloaded to C:\Users\User\AppData\Local\ms-playwright\ffmpeg-1011
Downloading WinLdd (playwright winLdd v1007) from https://cdn.playwright.dev/dbazure/download/playwright/builds/winLdd/1007/winLdd-win64.zip
0.1 MiB [=====] 100% 0.0s
WinLdd (playwright winLdd v1007) downloaded to C:\Users\User\AppData\Local\ms-playwright\winLdd-1007
Success! Created a Playwright Test project at C:\Windows\System32\ecommerce-assignment
Inside that directory, you can run several commands:

npx playwright test
  Runs the end-to-end tests.

npx playwright test --ui
  Starts the interactive UI mode.

npx playwright test --project=chromium
  Runs the tests only on Desktop Chrome.

npx playwright test example
  Runs the tests in a specific file.

npx playwright test --debug
  Runs the tests in debug mode.

npx playwright codegen
  Auto generate tests with Codegen.

We suggest that you begin by typing:

npx playwright test

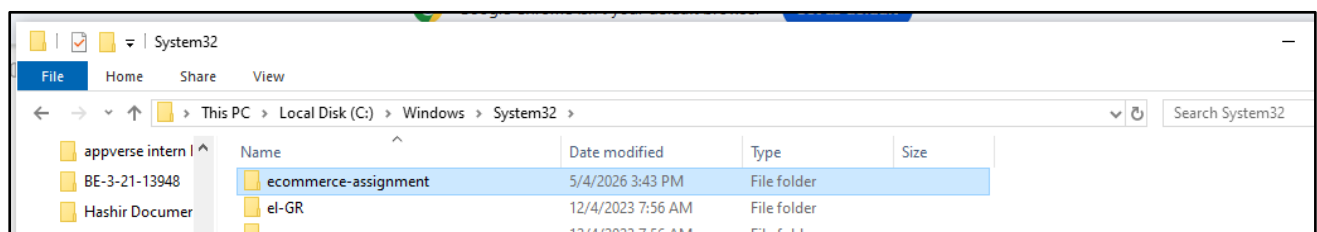
And check out the following files:
- .\tests\example.spec.js - Example end-to-end test
- .\playwright.config.js - Playwright Test configuration

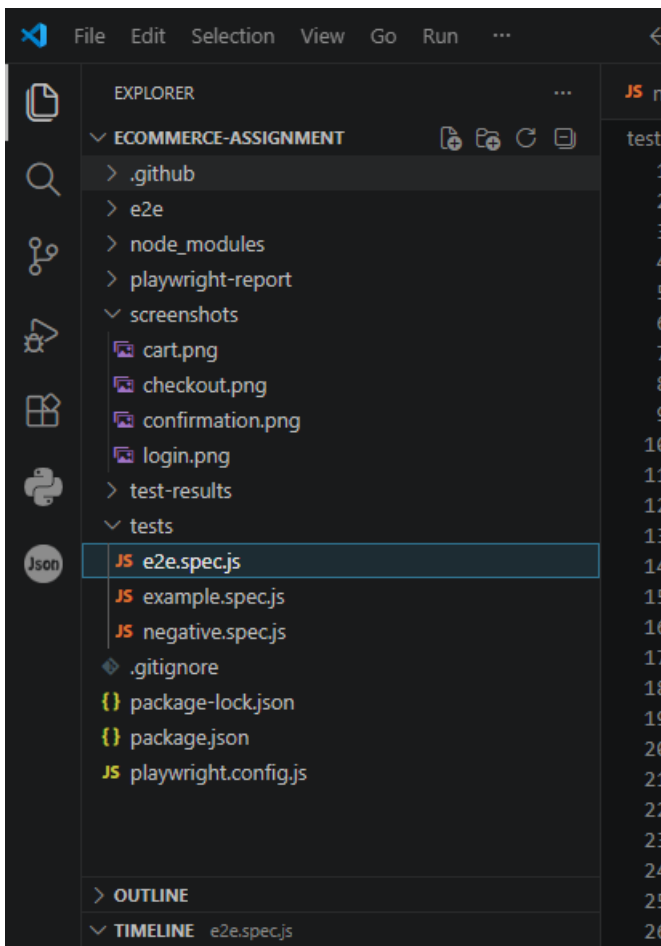
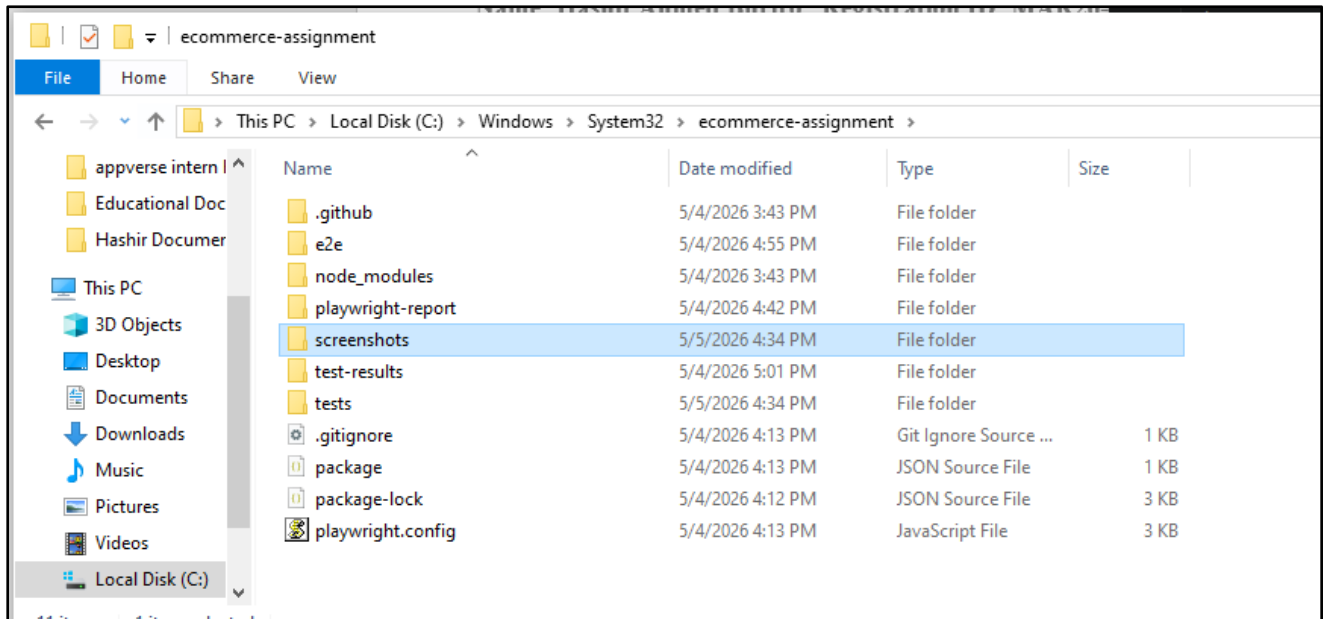
Visit https://playwright.dev/docs/intro for more information.

Happy hacking!

npm notice
npm notice New minor version of npm available! 11.12.1 -> 11.13.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.13.0
npm notice To update run: npm install -g npm@11.13.0
npm notice

C:\Windows\System32\ecommerce-assignment>
```





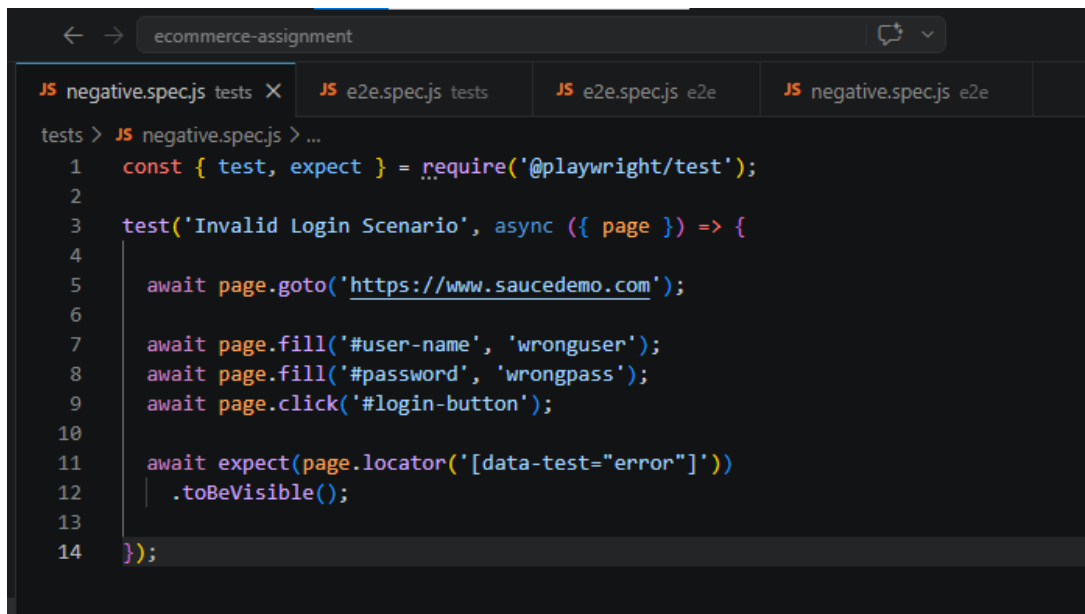
CODE:

Tests/e2e.spec.js:

```
ecommerce-assignment
JS negative.spec.js tests JS e2e.spec.js tests X JS e2e.spec.js e2e JS negative.spec.js e2e
tests > JS e2e.spec.js > test('E-commerce Checkout Flow Automation') callback
1  const { test, expect } = require('@playwright/test');
2
3  test('E-commerce Checkout Flow Automation', async ({ page }) => {
4
5      // 1. Open website
6      await page.goto('https://www.saucedemo.com');
7
8      // 2. Login
9      await page.fill('#user-name', 'standard_user');
10     await page.fill('#password', 'secret_sauce');
11     await page.click('#login-button');
12
13     await expect(page).toHaveURL(/inventory/);
14
15     // Screenshot login
16     await page.screenshot({ path: 'login.png' });
17
18     // 3. Add product to cart
19     await page.click('#add-to-cart-sauce-labs-backpack');
20     await page.click('.shopping_cart_link');
21
22     // Screenshot cart
23     await page.screenshot({ path: 'cart.png' });
24
25     // 4. Checkout
26     await page.click('#checkout');
27
28     await page.fill('#first-name', 'Hashir');
```

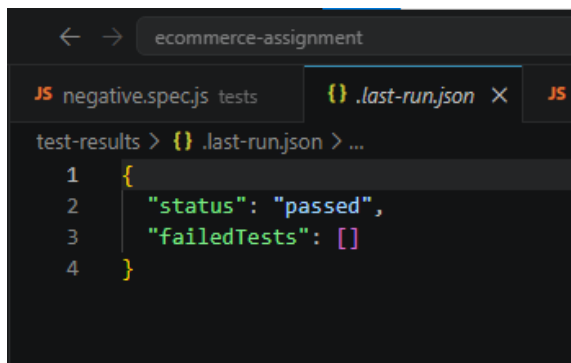
```
26     await page.click('#checkout');
27
28     await page.fill('#first-name', 'Hashir');
29     await page.fill('#last-name', 'Ahmed');
30     await page.fill('#postal-code', '74000');
31
32     await page.click('#continue');
33
34     // Screenshot checkout
35     await page.screenshot({ path: 'checkout.png' });
36
37     // 5. Finish order
38     await page.click('#finish');
39
40     // 6. Verify order
41     await expect(page.locator('.complete-header'))
42     .toHaveText('Thank you for your order!');
43
44     // Screenshot confirmation
45     await page.screenshot({ path: 'confirmation.png' });
46
47     });
```

Tests/negative.spec.js:

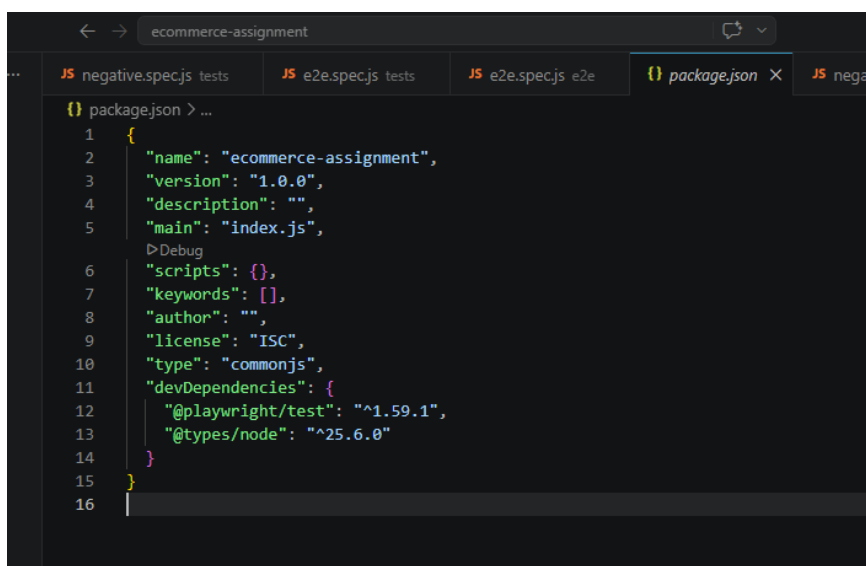


```
1 const { test, expect } = require('@playwright/test');
2
3 test('Invalid Login Scenario', async ({ page }) => {
4
5     await page.goto('https://www.saucedemo.com');
6
7     await page.fill('#user-name', 'wronguser');
8     await page.fill('#password', 'wrongpass');
9     await page.click('#login-button');
10
11     await expect(page.locator('[data-test="error"]'))
12         .toBeVisible();
13
14 });
```

Test result json:



```
1 {
2   "status": "passed",
3   "failedTests": []
4 }
```



```
1 {
2   "name": "ecommerce-assignment",
3   "version": "1.0.0",
4   "description": "",
5   "main": "index.js",
6   "scripts": {},
7   "keywords": [],
8   "author": "",
9   "license": "ISC",
10  "type": "commonjs",
11  "devDependencies": {
12    "@playwright/test": "^1.59.1",
13    "@types/node": "^25.6.0"
14  }
15 }
```

Playwright install

>npm init playwright@latest

```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.19045.7184]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>cd ecommerce-assignment

C:\Windows\System32\ecommerce-assignment>npm init playwright@latest

> ecommerce-assignment@1.0.0 npx
> create-playwright

Getting started with writing end-to-end tests with Playwright:
Initializing project in '.'
√ Do you want to use TypeScript or JavaScript? · JavaScript
√ Where to put your end-to-end tests? · e2e
√ Add a GitHub Actions workflow? (Y/n) · true
√ Install Playwright browsers (can be done manually via 'npx playwright install')? (Y/n) · true
Installing Playwright Test (npm install --save-dev @playwright/test)...

up to date, audited 6 packages in 3s

found 0 vulnerabilities
√ C:\Windows\System32\ecommerce-assignment\playwright.config.js already exists. Override it? (y/N) · false
√ C:\Windows\System32\ecommerce-assignment\.github\workflows\playwright.yml already exists. Override it? (y/N) · false
√ C:\Windows\System32\ecommerce-assignment\example.spec.js already exists. Override it? (y/N) · false
Writing package.json.
Downloading browsers (npx playwright install)...
Success! Created a Playwright Test project at C:\Windows\System32\ecommerce-assignment

Inside that directory, you can run several commands:

  npx playwright test
    Runs the end-to-end tests.

  npx playwright test --ui
    Starts the interactive UI mode.

  npx playwright test --project=chromium
    Runs the tests only on Desktop Chrome.

  npx playwright test example
    Runs the tests in a specific file.

  npx playwright test --debug
    Runs the tests in debug mode.
```

npx playwright test and Output:

```
C:\Windows\System32\ecommerce-assignment\playwright.config.js already exists. Override it? (y/N) · false
√ C:\Windows\System32\ecommerce-assignment\.github\workflows\playwright.yml already exists. Override it? (y/N) · false
√ C:\Windows\System32\ecommerce-assignment\example.spec.js already exists. Override it? (y/N) · false
Writing package.json.
Downloading browsers (npx playwright install)...
Success! Created a Playwright Test project at C:\Windows\System32\ecommerce-assignment

Inside that directory, you can run several commands:

  npx playwright test
    Runs the end-to-end tests.

  npx playwright test --ui
    Starts the interactive UI mode.

  npx playwright test --project=chromium
    Runs the tests only on Desktop Chrome.

  npx playwright test example
    Runs the tests in a specific file.

  npx playwright test --debug
    Runs the tests in debug mode.

  npx playwright codegen
    Auto generate tests with Codegen.

We suggest that you begin by typing:

  npx playwright test

And check out the following files:
- .\example.spec.js - Example end-to-end test
- .\playwright.config.js - Playwright Test configuration

Visit https://playwright.dev/docs/intro for more information.

Happy hacking!

C:\Windows\System32\ecommerce-assignment>
```



```
Select C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.7184]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32\ecommerce-assignment>code

C:\Windows\System32\ecommerce-assignment>npx playwright test
Error: EPERM: operation not permitted, unlink 'C:\Windows\System32\ecommerce-assignment\test-results\last-run.json'
Error in reporter Error: EPERM: operation not permitted, open 'C:\Windows\System32\ecommerce-assignment\playwright-report\index.html'
    at open (node:internal/fs/promises:640:25)
    at Object.writeFile (node:internal/fs/promises:1257:14)
    at HtmlBuilder.writeStaticAssets (C:\Windows\System32\ecommerce-assignment\node_modules\playwright\lib\reporters\html.js:330:7)
    at HtmlBuilder.build (C:\Windows\System32\ecommerce-assignment\node_modules\playwright\lib\reporters\html.js:292:29) at HtmlReporter.onEnd (C:\Windows\System32\ecommerce-assignment\node_modules\playwright\lib\reporters\html.js:128:25)
    at wrapAsync (C:\Windows\System32\ecommerce-assignment\node_modules\playwright\lib\reporters\multiplexer.js:101:12)
    at Multiplexer.onEnd (C:\Windows\System32\ecommerce-assignment\node_modules\playwright\lib\reporters\multiplexer.js:69:25)
    at InternalReporter.onEnd (C:\Windows\System32\ecommerce-assignment\node_modules\playwright\lib\reporters\internalReporter.js:81:12)
    at finishTaskRun (C:\Windows\System32\ecommerce-assignment\node_modules\playwright\lib\runner\tasks.js:94:26)
    at runTasks (C:\Windows\System32\ecommerce-assignment\node_modules\playwright\lib\runner\tasks.js:81:10)
    at runAllTestsWithConfig (C:\Windows\System32\ecommerce-assignment\node_modules\playwright\lib\runner\testRunner.js:388:18)
    at runTests (C:\Windows\System32\ecommerce-assignment\node_modules\playwright\lib\testActions.js:96:18)
    at i.<anonymous> (C:\Windows\System32\ecommerce-assignment\node_modules\playwright\lib\program.js:56:7) {
  errno: -4048,
  code: 'EPERM',
  syscall: 'open',
  path: 'C:\\Windows\\System32\\ecommerce-assignment\\playwright-report\\index.html'
}
Error in reporter Error: EPERM: operation not permitted, open 'C:\Windows\System32\ecommerce-assignment\test-results\last-run.json'
    at open (node:internal/fs/promises:640:25)
    at Object.writeFile (node:internal/fs/promises:1257:14)
    at LastRunReporter.onEnd (C:\Windows\System32\ecommerce-assignment\node_modules\playwright\lib\runner\lastRun.js:71:5)
    at wrapAsync (C:\Windows\System32\ecommerce-assignment\node_modules\playwright\lib\reporters\multiplexer.js:101:12)
    at Multiplexer.onEnd (C:\Windows\System32\ecommerce-assignment\node_modules\playwright\lib\reporters\multiplexer.js:69:25)
    at InternalReporter.onEnd (C:\Windows\System32\ecommerce-assignment\node_modules\playwright\lib\reporters\internalReporter.js:81:12)
    at finishTaskRun (C:\Windows\System32\ecommerce-assignment\node_modules\playwright\lib\runner\tasks.js:94:26)
    at runTasks (C:\Windows\System32\ecommerce-assignment\node_modules\playwright\lib\runner\tasks.js:81:10)
    at runAllTestsWithConfig (C:\Windows\System32\ecommerce-assignment\node_modules\playwright\lib\runner\testRunner.js:388:18)
    at runTests (C:\Windows\System32\ecommerce-assignment\node_modules\playwright\lib\testActions.js:96:18)
    at i.<anonymous> (C:\Windows\System32\ecommerce-assignment\node_modules\playwright\lib\program.js:56:7) {
  errno: -4048,
  code: 'EPERM',
  syscall: 'open',
  path: 'C:\\Windows\\System32\\ecommerce-assignment\\test-results\\last-run.json'
}

```

```
C:\Windows\System32\ecommerce-assignment>
12 passed (37.3s)
```

To open last HTML report run:

```
(c) Microsoft Corporation. All Rights Reserved.

C:\Windows\system32>cd ecommerce-assignment

C:\Windows\System32\ecommerce-assignment>npx playwright test

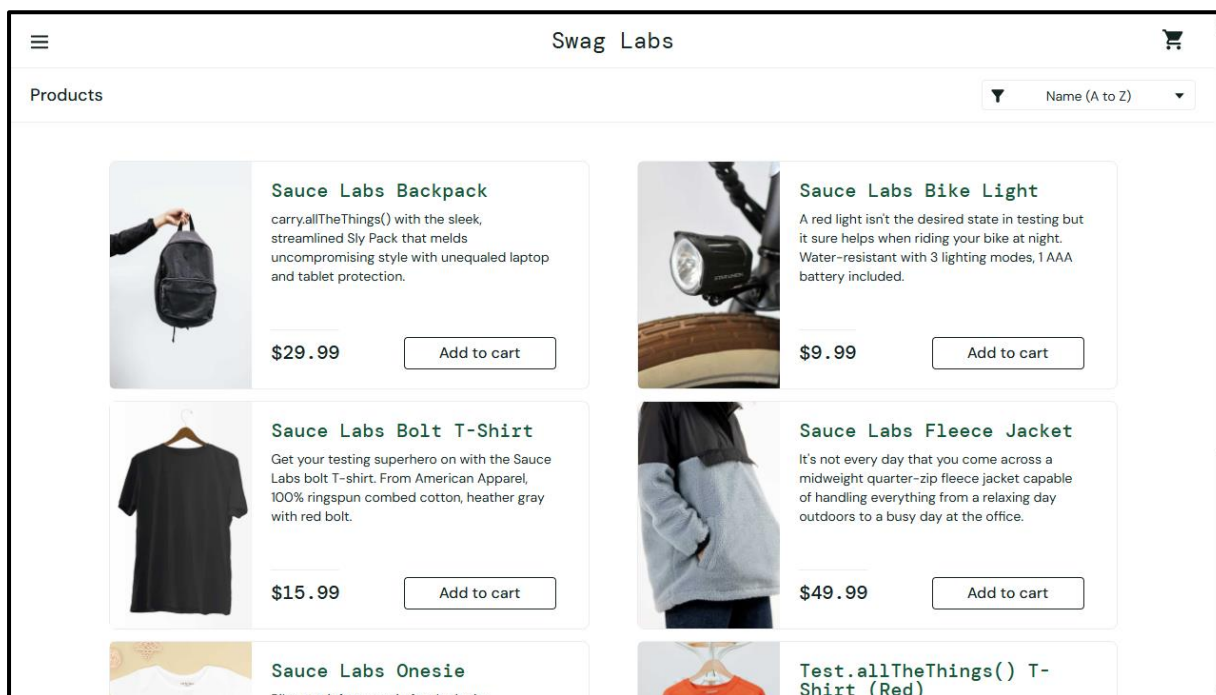
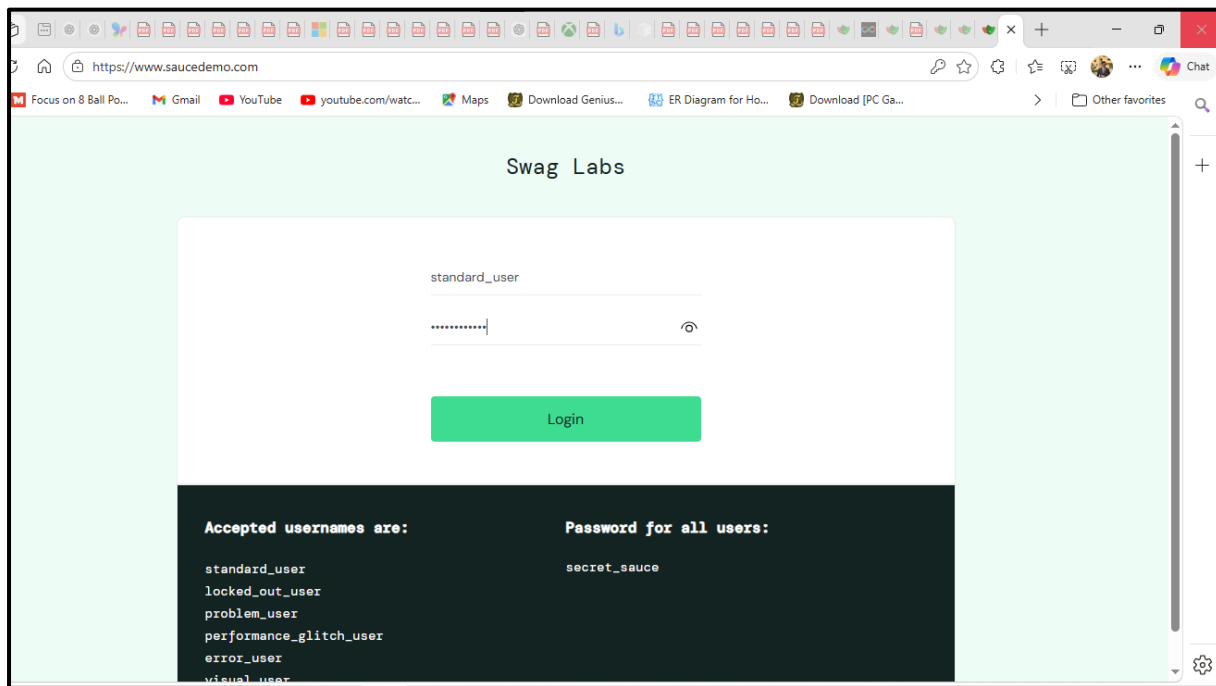
Running 12 tests using 2 workers
12 passed (37.9s)

To open last HTML report run:

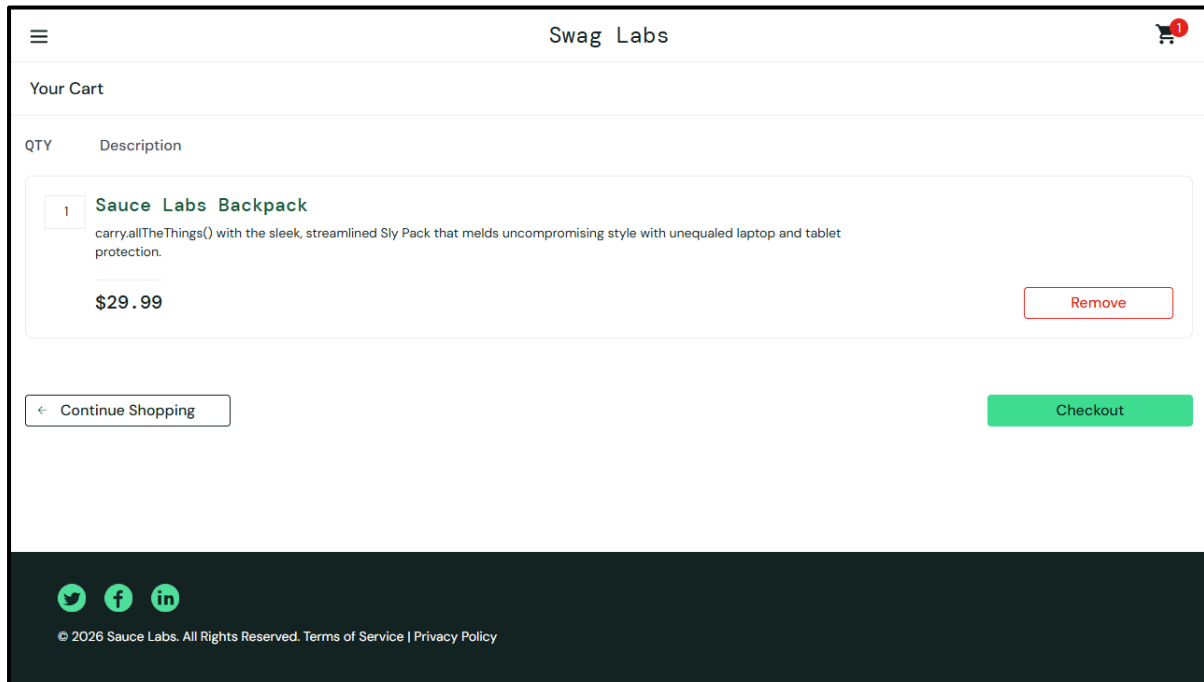
npx playwright show-report

```

Login Execution (Successfully Login test pass)



Add to Cart Execution (Successfully add to cart test pass)



Swag Labs

Your Cart

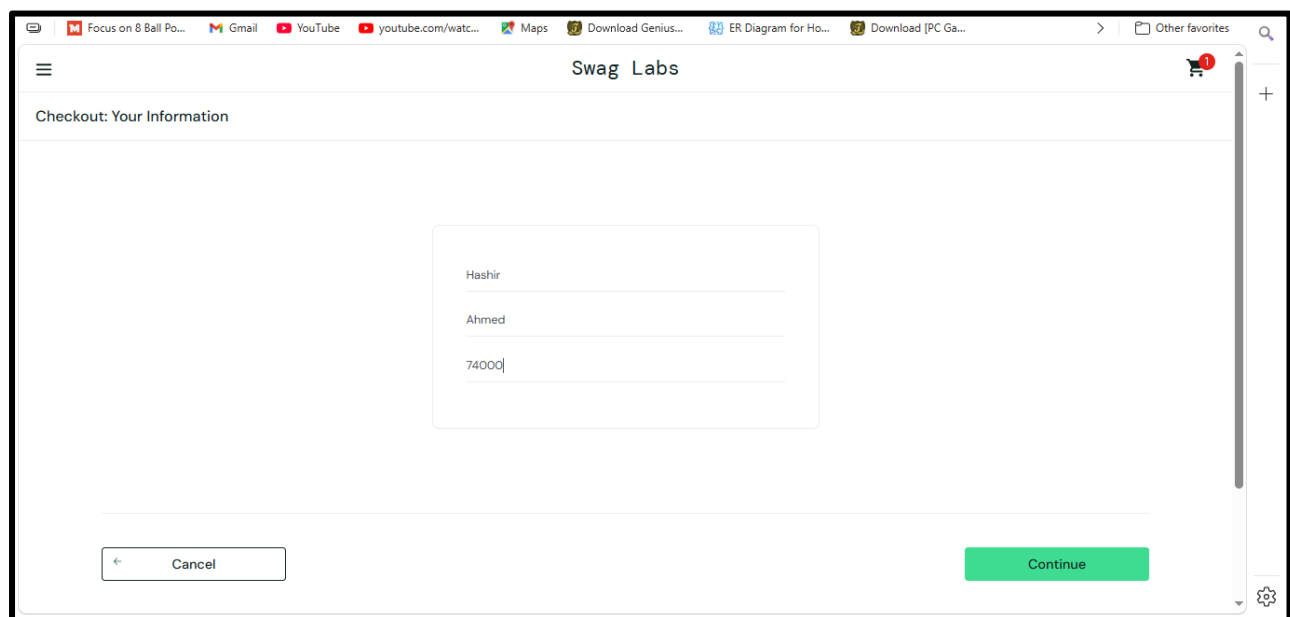
QTY	Description
1	Sauce Labs Backpack carry.allTheThings() with the sleek, streamlined Sly Pack that melds uncompromising style with unequaled laptop and tablet protection. \$29.99

[Remove](#)

[← Continue Shopping](#) [Checkout](#)

© 2026 Sauce Labs. All Rights Reserved. Terms of Service | Privacy Policy

Checkout Form Execution



Swag Labs

Checkout: Your Information

Hashir

Ahmed

74000

[← Cancel](#) [Continue](#)

Swag Labs

1

Checkout: Overview

QTY	Description
1	Sauce Labs Backpack carry.allTheThings() with the sleek, streamlined Sly Pack that melds uncompromising style with unequaled laptop and tablet protection. \$29.99

Payment Information:

SauceCard #31337

Shipping Information:

Free Pony Express Delivery!

Price Total

Item total: \$29.99

Tax: \$2.40

Total: \$32.39

Cancel

Finish

Order Confirmation

Swag Labs

Checkout: Complete!

Thank you for your order!

Your order has been dispatched, and will arrive just as fast as the pony can get there!

Back Home

© 2026 Sauce Labs. All Rights Reserved. Terms of Service | Privacy Policy

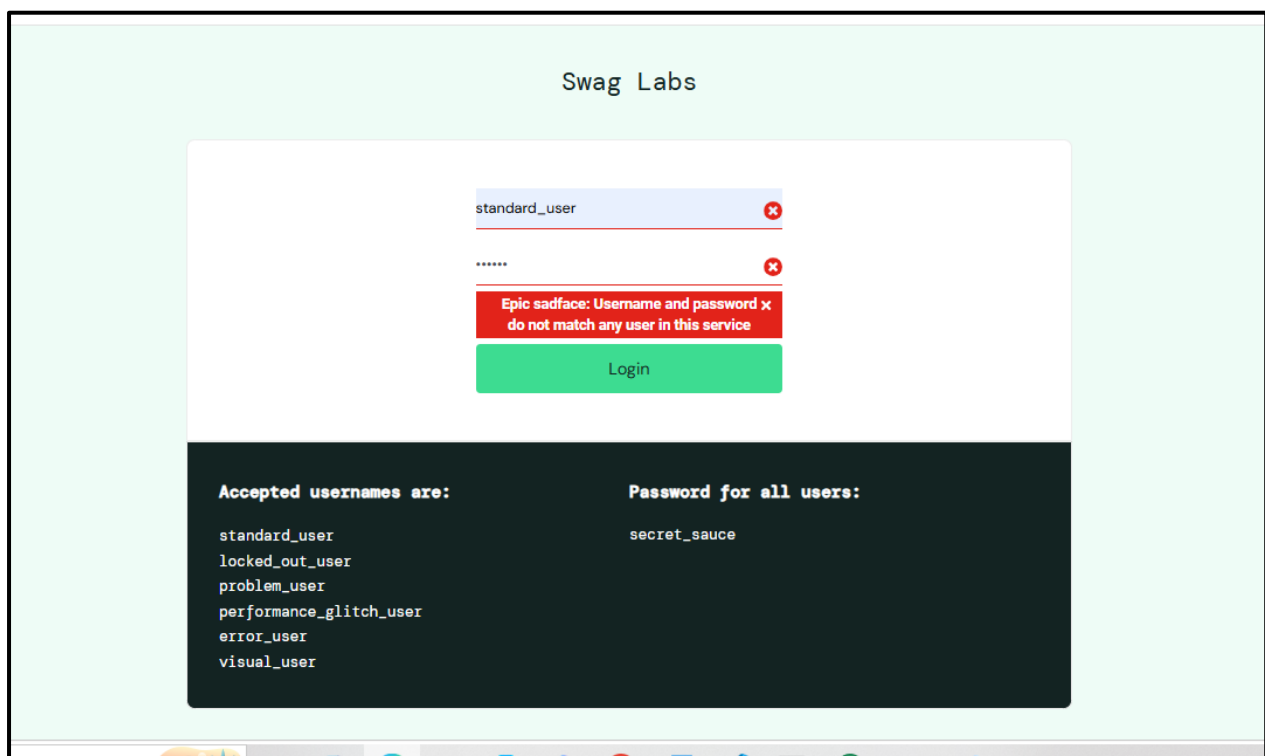
Negative Testing:

```
ecommerce-assignment

JS negative.spec.js tests X JS e2e.spec.js tests JS e2e.spec.js e2e JS negative.spec.js e2e

tests > JS negative.spec.js > ...
1  const { test, expect } = require('@playwright/test');
2
3  test('Invalid Login Scenario', async ({ page }) => {
4
5      await page.goto('https://www.saucedemo.com');
6
7      await page.fill('#user-name', 'wronguser');
8      await page.fill('#password', 'wrongpass');
9      await page.click('#login-button');
10
11     await expect(page.locator('[data-test="error"]'))
12       .toBeVisible();
13
14 });
```

Error message displayed for invalid login attempt

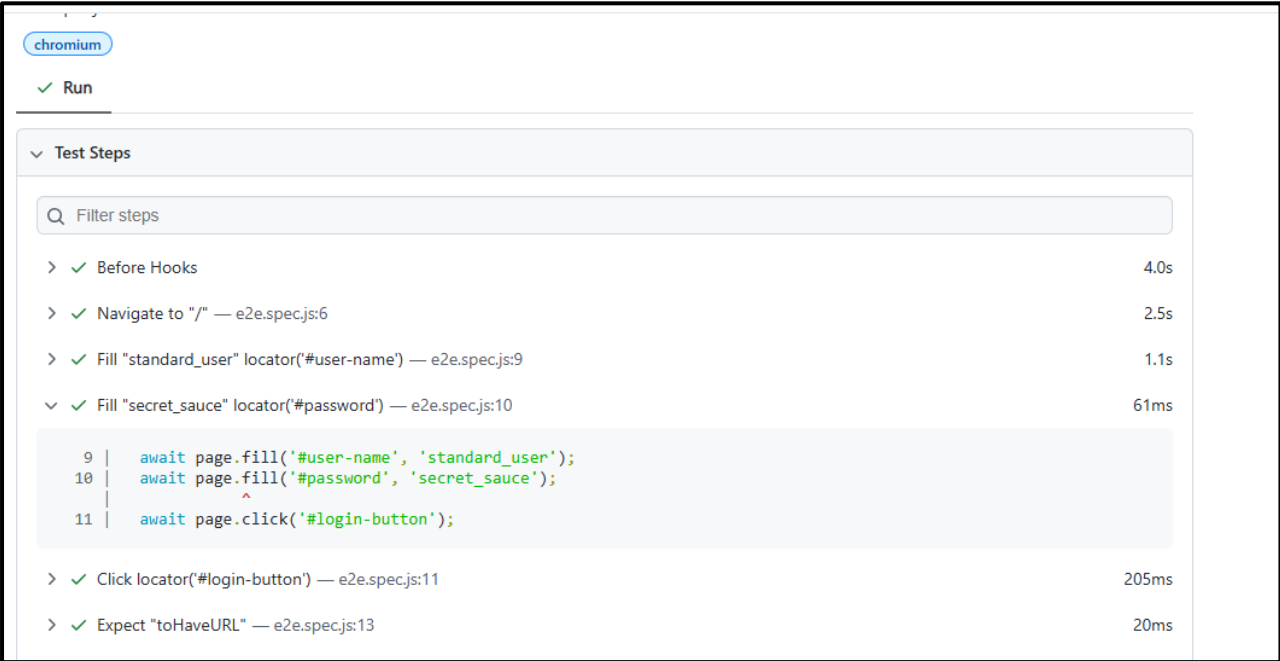
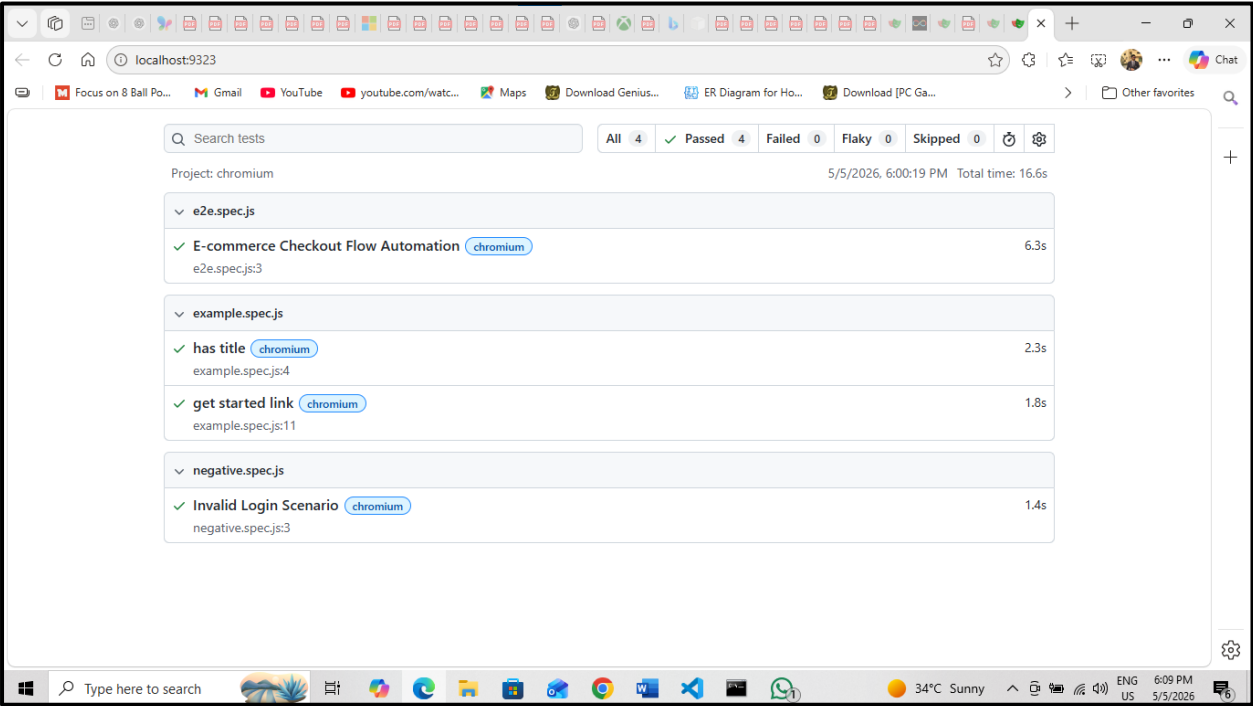


Test Report

Playwright test execution report showing all test results

```
C:\Windows\System32\ecommerce-assignment>npx playwright show-report

Serving HTML report at http://localhost:9323. Press Ctrl+C to quit.
```



✓ Click locator('#checkout') — e2e.spec.js:26

```
25 | // 4. Checkout
26 | await page.click('#checkout');
    |               ^
27 |
```

✓ Fill "Hashir" locator('#first-name') — e2e.spec.js:28

```
27 |
28 | await page.fill('#first-name', 'Hashir');
    |               ^
29 | await page.fill('#last-name', 'Ahmed');
```

✓ Fill "Ahmed" locator('#last-name') — e2e.spec.js:29

```
28 | await page.fill('#first-name', 'Hashir');
29 | await page.fill('#last-name', 'Ahmed');
    |               ^
30 | await page.fill('#postal-code', '74000');
```

✓ Fill "74000" locator('#postal-code') — e2e.spec.js:30

```
29 | await page.fill('#last-name', 'Ahmed');
30 | await page.fill('#postal-code', '74000');
    |               ^
31 |
```

✓ Click locator('#continue') — e2e.spec.js:32

```
31 |
32 | await page.click('#continue');
    |               ^
33 |
```

✓ Screenshot — e2e.spec.js:35

```
34 | // Screenshot checkout
35 | await page.screenshot({ path: 'checkout.png' });
    |               ^
36 |
```

> ✓ Click locator('#finish') — e2e.spec.js:38

✓ Expect "toHaveText" locator('.complete-header') — e2e.spec.js:42

```
41 | await expect(page.locator('.complete-header'))
42 |   .toHaveText('Thank you for your order!');
    |               ^
43 |
```

✓ Screenshot — e2e.spec.js:45

Invalid Login Scenario

negative.spec.js:3 1.4s

chromium

✓ Run

Test Steps

Q Filter steps

- > ✓ Before Hooks 239ms
- > ✓ Navigate to "/" — negative.spec.js:5 955ms
- > ✓ Fill "wronguser" locator("#user-name") — negative.spec.js:7 65ms
- > ✓ Fill "wrongpass" locator("#password") — negative.spec.js:8 26ms
- > ✓ Click locator("#login-button") — negative.spec.js:9 91ms
- > ✓ Expect "toBeVisible" locator("[data-test="error"]") — negative.spec.js:12 7ms
- > ✓ After Hooks 63ms

4 |

5 | await page.goto('https://www.saucedemo.com');

6 |

✓ Fill "wronguser" locator("#user-name") — negative.spec.js:7

6 |

7 | await page.fill('#user-name', 'wronguser');

8 | await page.fill('#password', 'wrongpass');

✓ Fill "wrongpass" locator("#password") — negative.spec.js:8

7 | await page.fill('#user-name', 'wronguser');

8 | await page.fill('#password', 'wrongpass');

9 | await page.click('#login-button');

✓ Click locator("#login-button") — negative.spec.js:9

8 | await page.fill('#password', 'wrongpass');

9 | await page.click('#login-button');

10 |

✓ Expect "toBeVisible" locator("[data-test="error"]") — negative.spec.js:12

11 | await expect(page.locator('[data-test="error"]'))

RESULT (Test Execution Result)

The automation test cases for the E-commerce checkout flow were executed successfully using Playwright. All major test scenarios including user login, product selection, adding items to cart, checkout process, and order confirmation were tested. The test execution completed without any critical failures. Positive test cases passed successfully, confirming that the system works as expected for valid user actions. Negative test cases also behaved correctly by displaying proper error messages for invalid inputs. Overall, the application demonstrated stable behavior across all tested modules, and the automation script successfully validated the end-to-end workflow.

Test Report Summary:

The generated Playwright HTML report shows that all test cases were executed successfully with a **100% pass rate**. A total of multiple test scenarios were executed including login validation, cart operations, checkout form handling, and order confirmation. No critical errors or test failures were observed during execution. The report confirms that the application is functioning correctly in both positive and negative scenarios. Screenshots captured during execution further validate each step of the workflow, providing clear evidence of successful automation testing.

CONCLUSION

This project successfully demonstrates end-to-end automation testing of an E-commerce checkout flow using Playwright. It helps in understanding how automation testing improves software quality by reducing manual effort, increasing accuracy, and ensuring faster execution of test cases. The project covers key functionalities such as login, product handling, checkout process, and order confirmation along with negative testing scenarios. Overall, automation testing proved to be an efficient approach for validating web application behavior and ensuring a smooth and reliable user experience.